
CO3 - AT2

DATA INTERPRETATION

Sampling Distributions, Covariance, Correlation, Causation & Missing Data

N HARSHA VARDHAN

Register No.: 192324189

Course: DSA0405 - Fundamentals of Data Sciences

Slot: D

Saveetha School of Engineering (SIMATS)

Question 6 — Sampling Distribution Reasoning

Given: Population mean $\mu = 500$, population SD $\sigma = 80$, sample size $n = 64$

(a) Standard Error of the Sample Mean

$$SE = \sigma / \sqrt{n} = 80 / \sqrt{64} = 80 / 8 = 10$$

The standard error of the sample mean is 10.

(b) $P(\text{Sample Mean} > 520)$

$$z = (\bar{x} - \mu) / SE = (520 - 500) / 10 = 2.0$$

From the standard normal table, $P(Z > 2.0) \approx 0.0228$. So there is roughly a 2.28% chance that a random sample of 64 observations produces a sample mean exceeding 520, assuming the sampling distribution of the mean is approximately normal (justified here by the Central Limit Theorem, since $n = 64$ is reasonably large).

(c) Effect of Increasing n to 256

$$SE (n=256) = 80 / \sqrt{256} = 80 / 16 = 5$$

Increasing the sample size from 64 to 256 (a 4× increase) shrinks the standard error from 10 to 5 — exactly half. This illustrates the inverse-square-root law governing sampling distributions: SE is proportional to $1/\sqrt{n}$, so quadrupling the sample size only halves the standard error (not quarters it). This is a direct consequence of the Central Limit Theorem — larger samples produce sample means that cluster more tightly around the true population mean, but with diminishing returns as n grows.

Python Verification

PYTHON

```
import numpy as np from scipy
import stats mu = 500      #
population mean
sigma = 80      # population standard deviation
n1 = 64      # sample size

# (a) Standard error of the sample mean se1
= sigma / np.sqrt(n1)
print("Standard Error (n=64):", se1)

# (b) P(sample mean > 520) z =
(520 - mu) / se1 p_exceeds = 1 -
stats.norm.cdf(z) print("z-
score:", z)
print("P(X-bar > 520):", round(p_exceeds, 4))

# (c) Standard error when n increases to 256 n2
= 256
se2 = sigma / np.sqrt(n2)
print("Standard Error (n=256):", se2)
print("SE ratio (se1/se2):", se1 / se2)
```

```
Standard Error (n=64): 10.0 z-score:
2.0
P(X-bar > 520): 0.0228
Standard Error (n=256): 5.0
SE ratio (se1/se2): 2.0
```

Question 7 — Covariance Calculation and Sign Interpretation

X - Advertising Spend (₹'000): 10, 20, 30, 40, 50

Y - Sales (₹ lakh): 15, 18, 25, 30, 42

$$\text{Cov}(X, Y) = \sum (x - \bar{x})(y - \bar{y}) / N = 660 / 5 = 132$$

Mean X	Mean Y	Cov(X,Y)	Pearson r
30.0	26.0	132.0	0.975

Direction

The covariance is positive (132), meaning advertising spend and sales move in the same direction — as ad spend rises, sales tend to rise too. This is fully consistent with intuition: more advertising should drive more sales.

What Magnitude Alone Fails to Tell You

The raw covariance value of 132 has no fixed scale of reference — it is expressed in the product of the two variables' units (₹'000 × ₹ lakh), so "132" carries no intuitive sense of how strong the relationship is, nor can it be compared against a covariance computed from a different pair of variables in different units. Correlation solves this by standardizing covariance against the two variables' standard deviations, producing a unit-free number between -1 and +1 (here, $r \approx 0.975$) that directly conveys both the strength and direction of the linear relationship — something magnitude alone cannot do. *Python*

Verification

PYTHON

```
import numpy as np

X = np.array([10, 20, 30, 40, 50]) # Advertising spend (Rs '000)
Y = np.array([15, 18, 25, 30, 42]) # Sales (Rs lakh)
mean_x, mean_y = np.mean(X), np.mean(Y)
cov_population = np.sum((X - mean_x) * (Y - mean_y)) / len(X)
correlation = np.corrcoef(X, Y)[0, 1]
print("Mean X:", mean_x, "Mean Y:", mean_y)
print("Population Covariance:", cov_population)
print("Pearson Correlation Coefficient:", round(correlation, 4))
```

Mean X: 30.0 Mean Y: 26.0
Population Covariance: 132.0
Pearson Correlation Coefficient: 0.9752

Question 8 — Correlation vs Causation Trap

Reported: Correlation between ice-cream sales and drowning incidents, $r = 0.85$

The Statistical Flaw

The newspaper commits the classic correlation-implies-causation fallacy. A strong correlation only shows that two variables move together — it says nothing about whether one causes the other, whether the causation runs the other way, or whether both are being driven by a third factor entirely.

The Likely Confounding Variable

Hot weather / summer season. During warmer months, more people buy ice cream and, independently, more people go swimming (increasing drowning risk). The season drives both variables simultaneously, creating a strong but entirely noncausal correlation between them.

What Additional Evidence Would Be Needed for Causation

- Controlling for the confounder — e.g. checking whether the correlation disappears once temperature/season is statistically controlled for (via regression or stratified analysis).
- Temporal precedence — evidence that the cause reliably occurs before the effect.
- A plausible causal mechanism — some biological or physical pathway explaining how ice cream consumption could directly cause drowning.
- Dose-response relationship — do drowning rates increase proportionally with ice cream sales even within the same season?
- Experimental or quasi-experimental evidence — ideally a randomized controlled trial or natural experiment (e.g. comparing regions/times with similar temperatures but different ice-cream consumption) that isolates the variable of interest.

Question 9 — Effect of Outliers on Correlation

Reported: $n = 9$, $r = 0.42$ (weak positive); after removing one extreme outlier, $r = 0.89$

What This Reveals About Pearson's Correlation

This swing from a weak (0.42) to a strong (0.89) correlation after removing a single point exposes how fragile Pearson's r can be in small samples. Pearson's correlation is built from means and squared deviations, so a single point that lies far from the rest of the data exerts disproportionate leverage on both the covariance and the variance terms in the formula. With only 9

points, one outlier represents over 11% of the sample and can single-handedly swing the whole statistic — a distortion that would matter far less in a large dataset where any one point carries little individual weight.

Would Spearman's Rank Correlation Be More Robust?

Yes. Spearman's correlation is computed on the ranks of the data rather than the raw values, so it only cares about relative ordering, not exact magnitude. An extreme outlier can only ever occupy the highest or lowest rank position — how far away it sits numerically is irrelevant to the rank-based calculation. This makes Spearman's rho far less sensitive to a single extreme value than Pearson's r, which is directly pulled by the outlier's exact distance from the mean. For a small, outlier-prone dataset like this one, Spearman's correlation would give a more stable, trustworthy picture of the underlying monotonic relationship.

Python Verification

PYTHON

```
from scipy import stats
import numpy as np

# Illustrative dataset: 8 points with a strong linear
pattern x8 = np.array([1, 2, 3, 4, 5, 6, 7, 8]) y8 =
np.array([3, 4, 5, 9, 7, 12, 11,
16])

# One extreme outlier added -> 9th point
x9 = np.append(x8, 9) y9 = np.append(y8,
2)

pearson_with = stats.pearsonr(x9, y9)[0]
pearson_without = stats.pearsonr(x8, y8)[0]
spearman_with = stats.spearmanr(x9, y9)[0]
spearman_without = stats.spearmanr(x8, y8)[0]

print("Pearson r (9 points, with outlier): ", round(pearson_with, 3))
print("Pearson r (8 points, outlier removed):", round(pearson_without, 3))
print("Spearman rho (9 points, with outlier): ", round(spearman_with, 3))
print("Spearman rho (8 points, outlier removed):", round(spearman_without, 3))

Pearson r (9 points, with outlier): 0.457
Pearson r (8 points, outlier removed): 0.946
Spearman rho (9 points, with outlier): 0.367
Spearman rho (8 points, outlier removed): 0.952
```

Question 10 — Data Preparation: Missing Values

Given: 200 rows, 15% missing in "Income"; missingness is more common among unemployed respondents

(a) Missing Data Mechanism

This is Missing At Random (MAR). The probability that "Income" is missing is not unrelated to the data (which would be MCAR — Missing Completely At Random), but it also isn't necessarily driven by the unobserved income value itself (which would point toward MNAR — Missing Not At Random). Instead, the missingness is systematically explained by another observed variable in the dataset — employment status — which is precisely the textbook definition of MAR: missingness depends on other observed variables, not on the missing value itself. (Note: if unemployed respondents were specifically hiding a low or zero income because of its value, the mechanism would shade toward MNAR — in practice this distinction can be hard to verify with certainty.)

(b) Why Mean Imputation Would Introduce Bias

Filling missing incomes with the overall sample mean assumes the missing values would, on average, resemble the rest of the dataset — but here they systematically come from unemployed respondents, who almost certainly have lower true incomes than the overall average. Imputing the overall mean would artificially inflate income figures for this subgroup, understating income inequality, distorting the income distribution's shape, shrinking its variance, and biasing any downstream analysis that relates income to employment status (the very relationship the missingness is tied to).

(c) A Better Strategy

- Conditional imputation — impute using the mean/median of Income within each employment-status subgroup, rather than the overall mean.
- Model-based / multiple imputation (e.g. MICE) — predict missing Income from employment status and other correlated variables, generating multiple plausible imputed datasets to properly reflect imputation uncertainty.

-
- Add a "missingness" indicator flag alongside any imputed value, so models can learn from the fact that a value was missing, not just its imputed substitute.
 - At minimum, report and analyze results separately by employment status to make the MAR-driven bias visible rather than papering over it with a single global imputation.